Heart Disease Prediction Using Machine Learning

DATA SCIENCE 2 FINAL PROJECT

BY: MATHEUS MAURICIO SANTOS GOMES
JOAQUIN HIDALGO
RAJAN JINADRA

Introduction

The problem we decided to address is whether we can predict if an individual has heart disease based on a set of medical and health-related factors. Heart disease is one of the most serious health conditions because early detection can help patients receive treatment sooner and reduce the risk of severe complications. For this project, we used a heart disease dataset containing patient information such as age, sex, blood pressure, cholesterol, maximum heart rate, chest pain type, and other health indicators.

The goal of this project was to build a classification model that predicts whether a patient is likely to have heart disease or not. The target variable in our dataset was target, where 0 represents no heart disease, and 1 represents the presence of heart disease. Since this is a binary classification problem, we used Random Forest, K-Nearest Neighbor, and Logistic Regression models to classify patients into one of the two categories. This project is useful because it shows how data science can support medical decision-making, although a model like this should not replace a doctor's diagnosis.

Dataset Description

The dataset used for this project is the **UCI Heart Disease Dataset (**https://www.kaggle.com/datasets/redwankarimsony/heart-disease-data **)** from Kaggle. It contains patient health information and a target variable that tells whether heart disease is present.

Some important features include:

Feature	**Meaning**
age	Patient age
sex	Patient sex

cp	Chest pain type
trestbps	Resting blood pressure
chol	Serum cholesterol
fbs	Fasting blood sugar
restecg	Resting ECG results
thalach	Maximum heart rate achieved
exang	Exercise-induced angina
oldpeak	ST depression caused by exercise
slope	Slope of the ST segment
ca	Number of major vessels
thal	Thalassemia result
target	Heart disease/no heart disease

The target variable is binary, where one class represents patients without heart disease and the other class represents patients with heart disease.

Data Cleaning:

Before building the models, the dataset needs to be cleaned and prepared. This is important because machine learning models need the data to be in a usable numerical format.

Data Inspection: The dataset was first loaded into Python using pandas and inspected to understand its structure, data types, and overall quality. Functions such as .info() and .describe() were used to identify potential issues, including missing values and incorrect data types.

Handling Missing Values: The dataset was checked for missing values. Any columns containing non-numeric entries were converted to numeric format, and missing values were handled appropriately to ensure the dataset was complete and usable for modeling.

Target Variable Transformation: The original target variable was not in binary format. Therefore, it was transformed into a binary variable where:

0 = No heart disease

1 = Presence of heart disease

This ensured the problem could be treated as a binary classification task.

Data Type Correction: Certain variables that were incorrectly stored as non-numeric (such as ca and thal) were converted into numeric format. This step ensured that all features were compatible with machine learning models.

Removing Irrelevant Features: The ID column was removed from the dataset because it is only an identifier and does not provide meaningful information for prediction. Including it could lead to misleading results, as the model might incorrectly treat it as an important feature.

Encoding Categorical Variables: Categorical variables such as chest pain type (cp), resting ECG (restecg), slope, and thalassemia were converted into numerical format using one-hot encoding. This allowed the model to interpret these features correctly. Binary variables such as sex and exercise-induced angina were already in numeric form and did not require additional encoding.

Removing Duplicates: The dataset was checked for duplicate rows, and any duplicates were removed to prevent bias in model training.

Checking Class Balance: The distribution of the target variable was analyzed and found to be relatively balanced, with a similar number of patients with and without heart disease. This ensured that evaluation metrics such as accuracy, precision, recall, and F1-score would be meaningful.

Final Dataset Preparation: After completing all preprocessing steps, the dataset consisted only of clean, numeric, and relevant features. It was then ready to be split into training and testing sets and used for model training.

Exploratory Data Analysis:

Before building the model, we performed exploratory data analysis to understand the structure, quality, and patterns within the dataset. The first step was checking the shape of the data, viewing the first few rows, identifying the column names, checking data types, finding missing values, and checking for duplicates.

The dataset had 920 rows and 16 columns, meaning there were 920 patient records available for analysis. The first few rows showed that the dataset included demographic information such as age and sex, as well as medical variables such as chest pain type, resting blood pressure, cholesterol, fasting blood sugar, resting ECG results, maximum heart rate, and exercise-induced angina

Categorical Summary:

	sex	dataset	cp	fbs	restecg	exang	slope	thal
count	920	920	920	830	918	865	611	434
unique	2	4	4	2	3	2	3	3
top	Male	Cleveland	asymptomatic	False	normal	False	flat	normal
freq	726	304	496	692	551	528	345	196

```
Numerical Columns:
Index(['id', 'age', 'trestbps', 'chol', 'thalch', 'oldpeak', 'ca', 'num',
       'heart_disease'],
      dtype='object')

Categorical Columns:
Index(['sex', 'dataset', 'cp', 'fbs', 'restecg', 'exang', 'slope', 'thal'], dtype='object')
```

```
Summary Statistics:
                id         age    trestbps        chol      thalch     oldpeak
count  920.000000  920.000000  861.000000  890.000000  865.000000  858.000000
mean   460.500000   53.510870  132.132404  199.130337  137.545665    0.878788
std    265.725422    9.424685   19.066070  110.780810   25.926276    1.091226
min      1.000000   28.000000    0.000000    0.000000   60.000000   -2.600000
25%    230.750000   47.000000  120.000000  175.000000  120.000000    0.000000
50%    460.500000   54.000000  130.000000  223.000000  140.000000    0.500000
75%    690.250000   60.000000  140.000000  268.000000  157.000000    1.500000
max    920.000000   77.000000  200.000000  603.000000  202.000000    6.200000

               ca         num  heart_disease
count  309.000000  920.000000     920.000000
mean     0.676375    0.995652       0.553261
std      0.935653    1.142693       0.497426
min      0.000000    0.000000       0.000000
25%      0.000000    0.000000       0.000000
50%      0.000000    1.000000       1.000000
75%      1.000000    2.000000       1.000000
max      3.000000    4.000000       1.000000
```

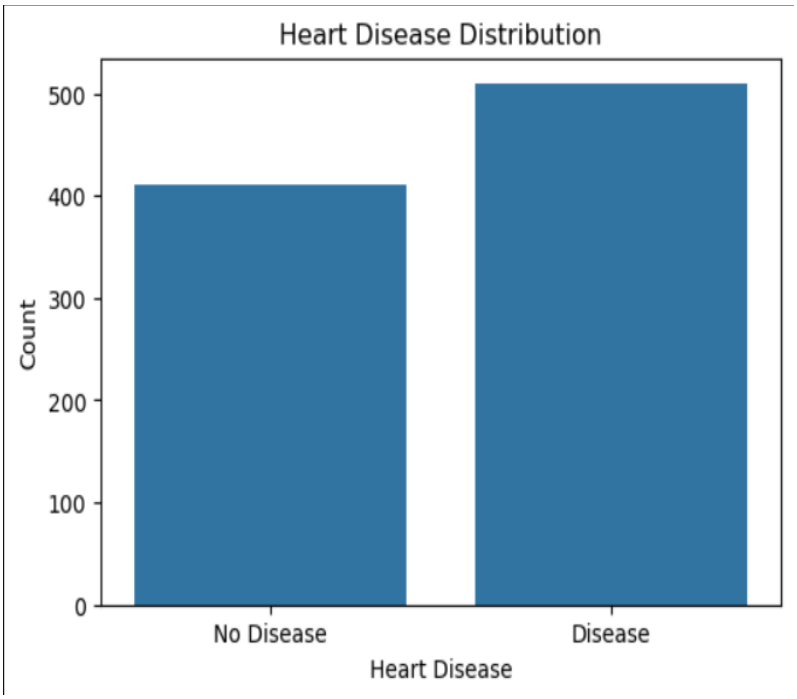
1. Target Distribution:

The target distribution plot shows the number of patients with and without heart disease. In this dataset, 509 patients had heart disease, while 411 patients did not have heart disease. This means that approximately 55.3% of the patients had heart disease, and 44.7% did not.

This is important because it shows that the dataset is fairly balanced. Since the two classes are not extremely uneven, the model is less likely to be heavily biased toward one class. This also means that accuracy, precision, recall, F1-score, and ROC-AUC are all useful metrics for evaluating the model.

```
Target Distribution:
heart_disease
1    509
0    411
Name: count, dtype: int64

Target Distribution Percentage:
heart_disease
1    55.326087
0    44.673913
Name: proportion, dtype: float64
```

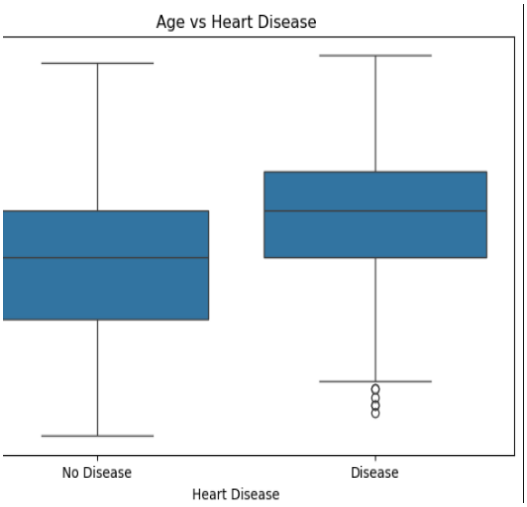


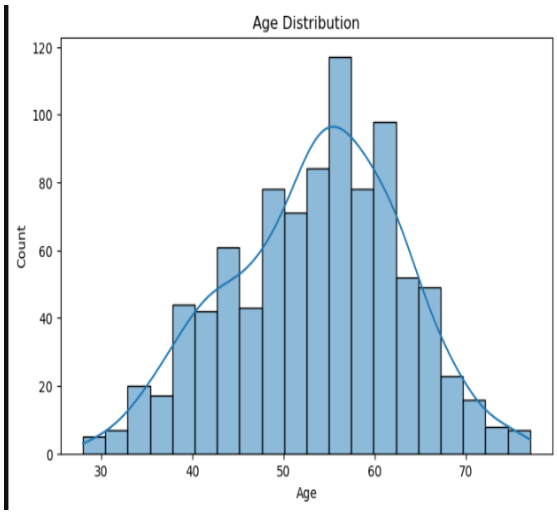
2. Age Distribution by Heart Disease:

The age distribution plot compares the ages of patients with and without heart disease. Age is an important feature because heart disease risk often increases as patients get older. This plot helps show whether heart disease cases are more common in certain age ranges.

From the dataset summary, the average patient age was about 53.5 years, with ages ranging from 28 to 77 years old. If the plot shows more heart disease cases in older age groups, this supports the idea that age may be an important predictor in the model. This plot is useful

because it allows. us to visually compare whether the heart disease group and the no-heart-disease group have different age patterns



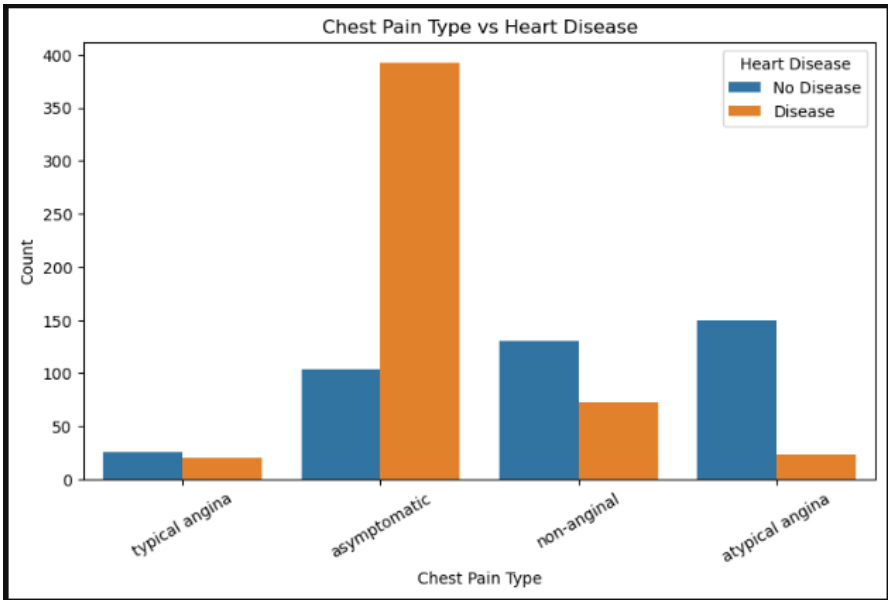


3. Chest Pain Type vs. Heart Disease:

The chest pain type plot compares different types of chest pain with the heart disease outcome. Chest pain type is an important medical variable because certain types of chest pain may be more strongly associated with heart disease.

In the dataset, the cp column included four chest pain categories. The most common chest pain category was asymptomatic, with 496 patients in that group. This plot helps show how heart disease cases are distributed across the different chest pain types.

This feature is important for the model because patients with certain chest pain patterns may be more likely to have heart disease. Therefore, chest pain type can help the KNN model separate patients with heart disease from those without heart disease.

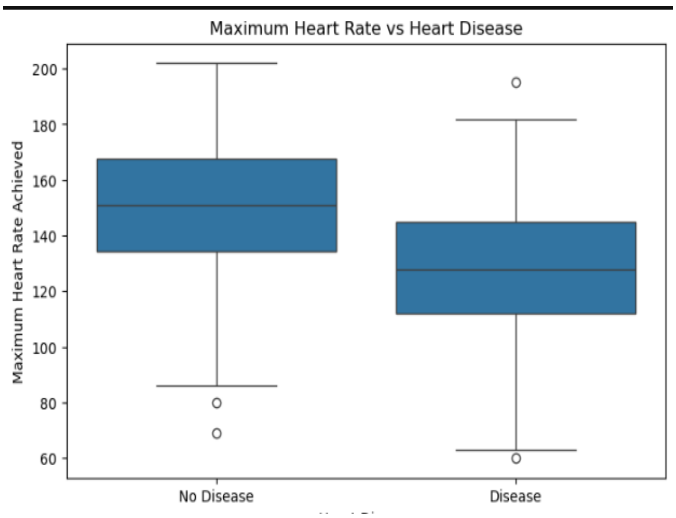


4. Maximum Heart Rate by Heart Disease:

The maximum heart rate plot compares the thalch variable across patients with and without heart disease. The thalch variable represents the maximum heart rate achieved by the patient.

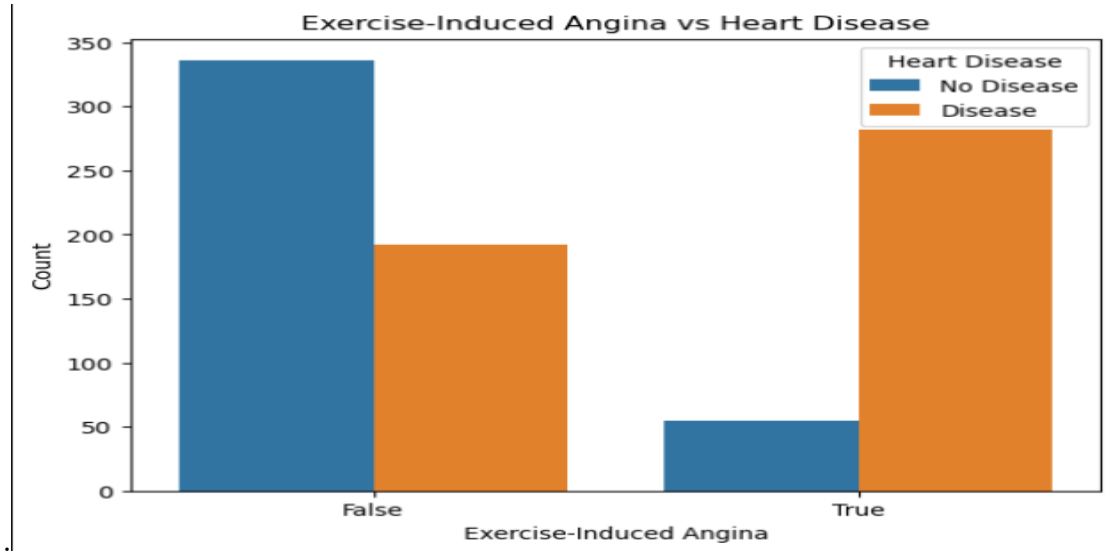
From the dataset summary, the average maximum heart rate was about 137.5, with values ranging from 60 to 202. This plot is useful because the maximum heart rate can show differences in cardiovascular function between patients with heart disease and those without heart disease.

If the two groups have noticeably different distributions, this suggests that maximum heart rate may be an important predictor for the KNN model. Since KNN uses the distance between patient records, differences in numerical features like maximum heart rate can influence how patients are classified.



5. Exercise-Induced Angina vs. Heart Disease:

The exercise-induced angina plot compares the exang variable with heart disease status. Exercise-induced angina refers to chest pain that occurs during physical activity. This is medically relevant because chest pain during exercise may indicate reduced blood flow to the heart. In the dataset, the most common value for exang was False, meaning many patients did not report exercise-induced angina. However, comparing this variable against the target helps show whether patients with exercise-induced angina were more likely to have heart disease. This plot is important because it provides a clear visual comparison between a symptom-related variable and the heart disease outcome. It can help explain why exang may be useful in predicting heart disease.

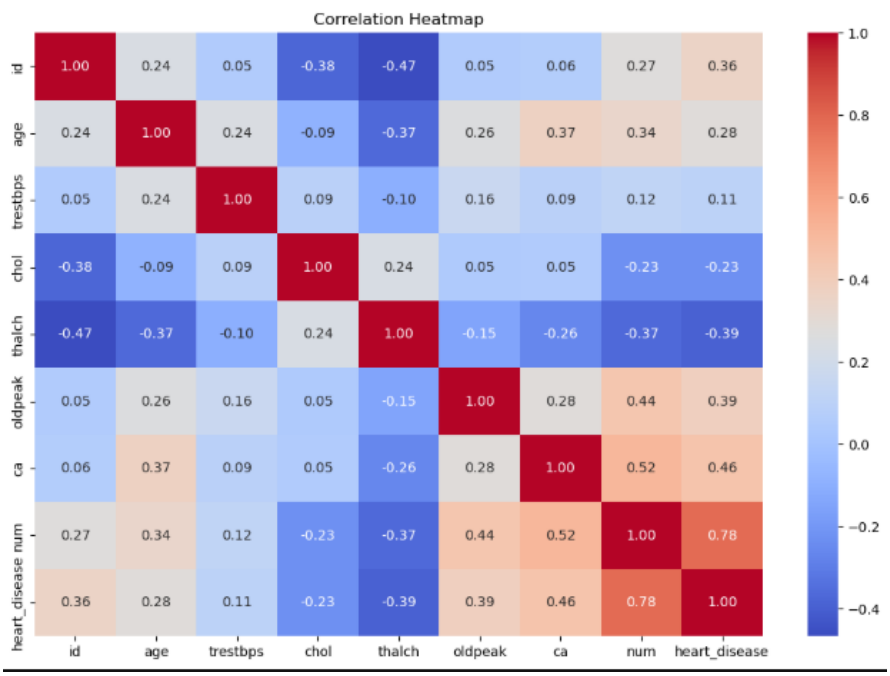


6. Correlation Heatmap:

The correlation heatmap shows the relationships between the numerical variables in the dataset. Correlation values range from -1 to 1. A positive correlation means that two variables tend to increase together, while a negative correlation means that one variable tends to decrease as the other increases.

This plot is useful because it helps identify which numerical features may be related to heart disease. For example, variables like age, resting blood pressure, cholesterol, maximum heart rate, oldpeak, and ca can be compared with the target variable to see whether they have positive or negative relationships with heart disease.

The heatmap also helps check whether any features are strongly correlated with each other. If two variables are highly correlated, they may contain similar information. For KNN, this matters because highly related or repeated features can influence distance calculations.



Definitions:

- Classification Model: A classification model is a machine learning method used to predict which category an observation belongs to. In this project, the model predicted whether a patient did not have heart disease, represented by 0, or did have heart disease, represented by 1.
- Accuracy: Accuracy measures how often the model made the correct prediction overall. It is calculated by dividing the number of correct predictions by the total number of predictions. Accuracy is useful when the dataset is fairly balanced, like our heart disease dataset.
- Precision: Precision measures how often the model was correct when it predicted heart disease. It is calculated as the number of true positives divided by the total number of predicted positives. In this project, precision is important because it shows how reliable the model is when it says a patient has heart disease.
- Recall: Recall, also called sensitivity, measures how well the model identifies actual heart disease cases. It is calculated as the number of true positives divided by the total number of actual positives. In this project, recall is very important because missing a patient who actually has heart disease could be risky in a medical setting.
- F1-Score: The F1-score combines precision and recall into one metric. It is useful when we want to balance the model's ability to correctly predict heart disease while also avoiding too many incorrect positive predictions. A higher F1-score means the model has a better balance between precision and recall.
- Confusion Matrix: A confusion matrix is a table that shows how many predictions the model got right and wrong. It includes true positives, true negatives, false positives, and false negatives. This is useful because it shows not just the overall performance of the model, but also what types of mistakes the model made.
- True Positive: A true positive occurs when the model correctly predicts that a patient has heart disease.
- True Negative: A true negative occurs when the model correctly predicts that a patient does not have heart disease.
- False Positive: A false positive occurs when the model predicts that a patient has heart disease, but the patient actually does not. This could lead to unnecessary follow-up testing.
- False Negative: A false negative occurs when the model predicts that a patient does not have heart disease, but the patient actually does. In this project, false negatives are especially important because they could mean missing a patient who needs medical attention.
- ROC-AUC: ROC-AUC measures how well the model separates the two classes across different decision thresholds. A higher ROC-AUC score means the model is better at distinguishing between patients with heart disease and patients without heart disease.

- K-Nearest Neighbors Model (KNN): K-Nearest Neighbors is a supervised machine learning algorithm used for classification. It classifies a new data point by looking at the k closest data points in the training set. The model then assigns the new point to the class that appears most often among those neighbors. In this project, KNN classified patients as either having heart disease or not having heart disease based on how similar they were to other patients.
- GridSearchCV: GridSearchCV is a method used to tune a model's hyperparameters. It tests multiple combinations of settings and chooses the best combination based on model performance. In this project, GridSearchCV was used to test different values of k, distance metrics, and weighting methods for the KNN model.
- Feature Scaling: Feature scaling is the process of putting numerical variables on a similar scale. This is especially important for KNN because the model uses distance to compare observations. If features are not scaled, variables with larger values, such as cholesterol or blood pressure, could have more influence than smaller-scale variables.
- StandardScaler: StandardScaler is a preprocessing method that standardizes numerical features so that they have a mean of 0 and a standard deviation of 1. In this project, StandardScaler was used before fitting the KNN model because KNN depends heavily on distance calculations.
- Logistic Regression Model: Logistic Regression is a supervised machine learning model used for binary classification problems. It predicts the probability that an observation belongs to a certain class. In this project, Logistic Regression predicts the probability that a patient has heart disease. If the probability is above a certain threshold, usually 0.5, the model classifies the patient as having heart disease. If the probability is below that threshold, the model classifies the patient as not having heart disease. Logistic Regression is useful because it is simple, fast, and easy to interpret.
- Random Forest Model: Random Forest is a supervised machine learning model that uses many decision trees to make a prediction. Each decision tree makes its own prediction, and the Random Forest combines the results to choose the final class. For classification, the final prediction is usually based on the majority vote of all the trees. In this project, Random Forest predicts whether a patient has heart disease or not based on patterns in the medical features. Random Forest is useful because it can handle complex relationships in the data and is less likely to overfit than a single decision tree.

Models Used

Model 1: Logistic Regression

Logistic Regression is a simple and interpretable classification model. It predicts the probability that a patient has heart disease. This model is useful because it can show how each feature affects the chance of heart disease.

For example, features like chest pain type, maximum heart rate, and number of major vessels may strongly influence the prediction.

Why use it?

Logistic Regression is a good baseline model because it is simple, fast, and easy to explain.

Process?

The workflow began by loading the dataset into a pandas DataFrame and performing exploratory data analysis, covering data types, summary statistics, class distribution, and missing value checks. Features were then separated from the target variable, with numerical and categorical columns identified automatically. After an 80/20 stratified train/test split, a ColumnTransformer was defined to apply StandardScaler to numerical columns and OneHotEncoder to categorical ones, both wrapped inside a scikit-learn Pipeline alongside a LogisticRegression estimator to prevent data leakage. A baseline model was first trained with default parameters to establish benchmark accuracy and ROC-AUC scores. Hyperparameter tuning was then performed using GridSearchCV with 5-fold stratified cross-validation, searching over regularisation strength (C), penalty type (L1 and L2), solver, and class weighting, optimising for ROC-AUC. The best pipeline was used to generate predictions on the held-out test set, and its performance was compared against the baseline via a classification report, confusion matrix, and overlaid ROC curves. Finally, feature coefficients and odds ratios were extracted from the fitted model and visualised to interpret the contribution of each feature to the classification outcome.

Results:

Metric	Baseline	Tuned	Delta
Accuracy	0.8500	0.850	0.0000
ROC-AUC	0.9308	0.942	0.0112

Classification Report (tuned model):

	precision	recall	f1-score	support
0	0.83	0.91	0.87	32
1	0.88	0.79	0.83	28
accuracy			0.85	60
macro avg	0.85	0.85	0.85	60
weighted avg	0.85	0.85	0.85	60

The tuned model achieved an accuracy of 85% on the held-out test set, matching the baseline exactly, though hyperparameter tuning did yield a meaningful improvement in ROC-AUC from 0.9308 to 0.9420 (+0.0112), indicating the optimized model ranks positive and negative cases more confidently, even if the decision boundary itself didn't shift. Looking at the classification report, the model performs slightly better at identifying class 0 (recall of 0.91) than class 1 (recall of 0.79), meaning it misses more true positives in class 1 worth investigating if class 1 represents the outcome of greater business interest, such as churn or fraud. Precision is reasonably balanced at 0.83 and 0.88 for classes 0 and 1, respectively. The macro and weighted averages align closely at 0.85 across precision, recall, and F1, suggesting the model generalizes consistently across both classes with no strong bias toward the majority class

Model 2: K-Nearest Neighbors

We used K-Nearest Neighbors, also known as KNN, as one of our classification models. KNN is a supervised machine learning algorithm that classifies a new data point by comparing it to the closest data points in the training dataset. The model then assigns the new point to the class that appears most often among its nearest neighbors.

In this project, we used KNN to predict whether a patient has heart disease or does not have heart disease. The model is comparing a patient's health information, such as age, cholesterol, blood pressure, maximum heart rate, and chest pain type, to other patients in the dataset. If the patient is most similar to patients who have heart disease, then the model is predicting heart disease. If the patient is more similar to patients who do not have heart disease, then the model is predicting no heart disease.

Why are we using it?

We used KNN because it is simple, easy to understand, and useful for classification problems. Since our dataset contains patient health measurements, KNN is helping us classify patients based on how similar their medical information is to that of other patients.

However, KNN depends heavily on feature scaling because it uses distance to compare data points. For example, cholesterol values have a much larger scale than variables like sex or fasting blood sugar. Because of this, we are standardizing the data before training the model so that no single feature has too much influence on the distance calculations.

Train-Test Split:

Before training the model, we split the dataset into training and testing sets using an 80/20 split. We are using the training set to fit the KNN model, while we are using the testing set to evaluate how well the model is performing on unseen data.

We are also using stratified sampling so that the training and testing sets keep a similar balance of patients with and without heart disease. This is helping make the model evaluation more reliable.

Model Training:

We trained the KNN model using a pipeline that includes feature scaling with StandardScaler and the KNN classifier. Feature scaling is important because KNN depends on distance calculations. Without scaling, features with larger numerical values could dominate the model.

To improve the model, we used GridSearchCV to tune the KNN hyperparameters. GridSearchCV tests multiple combinations of parameters and selects the best combination based on cross-validation accuracy.

The parameters tested include:

n_neighbors: 1 through 30

weights: uniform and distance

metric: Euclidean and Manhattan

The n_neighbors parameter controls how many nearby patients the model is using to make a prediction. The weights parameter determines whether all neighbors are being treated equally or whether closer neighbors have more influence. The metric parameter controls how distance is measured between patients.

KNN Model Performance:

After training and tuning the KNN model, we evaluated its performance on the testing data. The model produced the following results:

```
KNN Model Performance:
Accuracy: 0.8333
Precision: 0.875
Recall: 0.75
F1 Score: 0.8077
ROC-AUC: 0.9319

Confusion Matrix:
[[29  3]
 [ 7 21]]
```

The model achieved an accuracy of **83.33%**, meaning it correctly classified about 83% of the patients in the test set. This shows that the model performed well overall.

The precision score is **87.5%**, which means that when the model predicted a patient had heart disease, it was correct most of the time. This is a strong result because it shows that the model did not produce many false positives.

The recall score is **75%**, meaning the model correctly identified 75% of the actual heart disease cases. However, this also means that the model missed 25% of the patients who actually had heart disease. In a medical setting, this is important because false negatives can be dangerous. Missing a patient who actually has heart disease could delay treatment.

The F1-score is **80.77%**, showing that the model had a good balance between precision and recall.

The ROC-AUC score is **93.19%**, which is very strong. This indicates that the model was very good at separating patients with heart disease from patients without heart disease.

The Confusion Matrix shows that the model correctly predicted **29 patients without heart disease** and **21 patients with heart disease**. It incorrectly predicted **3 patients as having heart disease when they did not**, and it missed **7 patients who actually had heart disease**.

The model is better at identifying patients without heart disease than identifying patients with heart disease. This can also be seen in the classification report, where class 0 had a recall of **0.91**, while class 1 had a recall of **0.75**.

```
Classification Report:
              precision    recall  f1-score   support

           0       0.81      0.91      0.85        32
           1       0.88      0.75      0.81        28

    accuracy                           0.83        60
   macro avg       0.84      0.83      0.83        60
weighted avg       0.84      0.83      0.83        60
```

KNN Conclusion

The KNN model performed well overall, achieving an accuracy of **83.33%** and a strong ROC-AUC score of **93.19%**. The model also has a high precision score, meaning that when it is predicting heart disease, it is usually correct.

However, the recall score is lower at **75%**, meaning the model is missing some patients who have heart disease. This is an important limitation because, in healthcare, false negatives can be more serious than false positives. Missing a patient with heart disease could delay diagnosis or treatment.

Overall, found that KNN is a useful model for this project because it is easy to interpret and performs well on the dataset. However, compared to a model like Random Forest, KNN may be less reliable for medical prediction because it depends heavily on feature scaling, distance calculations, and the choice of k.

Model 3: Random Forest

Introduction

Random Forest is an ensemble machine learning model that combines multiple decision trees to improve prediction accuracy and reduce overfitting. Each tree is trained on a random subset of the data and features, and the final prediction is made by majority vote across all trees.

Why use it?

This model was chosen for this project because it can capture complex, non-linear relationships among variables and typically performs well on classification problems. Additionally, Random Forest provides feature importance, which helps identify the most influential factors in predicting heart disease.

Train-Test Split

Before training the model, the dataset was divided into training and testing sets using an 80/20 split. The training set was used to build the model, while the testing set was used to evaluate its performance on unseen data. Stratified sampling was applied to preserve the original distribution of the target variable across both sets.

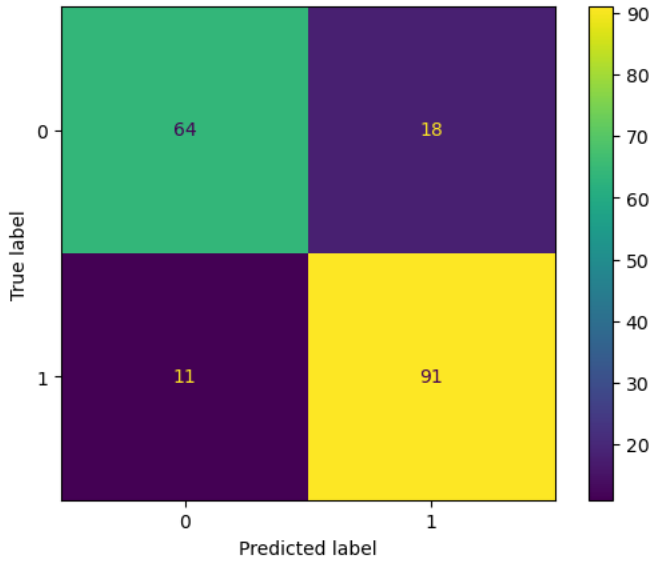
Model Training

The Random Forest classifier was trained using 100 decision trees (n_estimators = 100) with a fixed random state (random_state = 42) to ensure reproducibility. Random Forest was not tuned with GridSearchCV in this version; we used a baseline Random Forest with 100 trees to compare performance. Default parameters were used for the remaining hyperparameters, allowing the model to grow trees without a maximum depth limit and using the Gini impurity criterion for splitting.

Model Performance Metrics

The accuracy indicates that 84.2% of predictions were correct. Precision shows that when the model predicts heart disease, it is correct about 83.4% of the time. Recall is particularly important in this context, as it measures how well the model identifies actual cases of heart disease. A recall of 89.2% indicates that the model successfully detects most patients with the condition. The F1-score of 0.862 reflects a strong balance between precision and recall, confirming that the model performs consistently across both metrics.

Confusion Matrix Analysis



The confusion matrix provides a detailed breakdown of prediction outcomes:

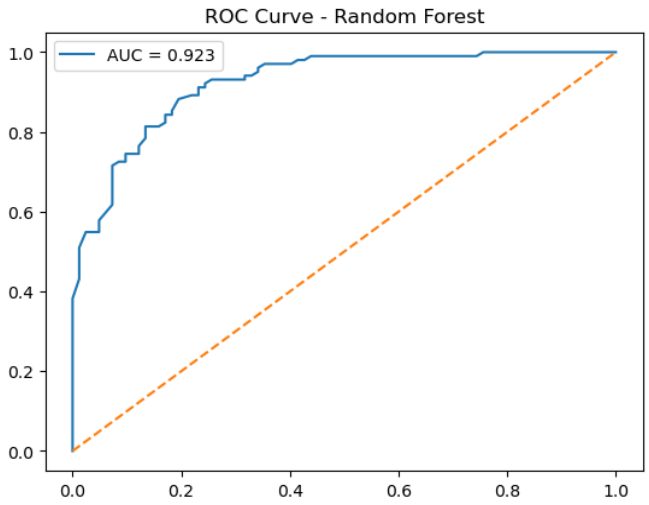
- True Negatives (TN): 64
- False Positives (FP): 18
- False Negatives (FN):11
- True Positives (TP): 91

The model produced only 11 false negatives, meaning very few patients with heart disease were incorrectly classified as healthy. This is a critical result in a medical application, where missing a diagnosis could have serious consequences.

ROC Curve and AUC Score

The Receiver Operating Characteristic (ROC) curve illustrates the trade-off between the true positive rate and false positive rate across different classification thresholds. The model achieved an AUC score of 0.953, indicating excellent ability to distinguish between patients with and without heart disease. An AUC value close to 1.0 suggests that the model performs significantly better than random guessing and has strong classification capability.

Here is the plot:



Feature Importance Analysis

The Random Forest model provides feature importance scores that highlight the most influential variables in predicting heart disease, with cholesterol (chol), maximum heart rate (thalach), age, and ST depression (oldpeak) emerging as the most important. These variables are medically relevant and strongly associated with cardiovascular health, as higher cholesterol levels and abnormal heart rate responses are well-known risk factors for heart disease, while the significance of oldpeak suggests that exercise-induced changes in heart activity play an important role in prediction. Overall, the feature importance results support the validity of the model, as the most influential features align with established clinical indicators of heart disease.

Random Forest Conclusion

The Random Forest model proved to be an effective approach for predicting heart disease, achieving strong performance across multiple metrics, including high accuracy and an AUC of 0.953. Most importantly, the model demonstrated a high recall, meaning it successfully identified the majority of patients with heart disease while minimizing false negatives, which is critical in a healthcare setting.

Additionally, feature importance analysis highlighted key variables such as cholesterol, maximum heart rate, age, and ST depression as the most influential predictors, aligning with known clinical risk factors. Overall, the Random Forest model showed strong predictive capability and reliability for this classification task.

Real-World Application

This project is useful because heart disease prediction can help support early screening and medical decision-making. Our models use patient health information such as age, cholesterol, resting blood pressure, maximum heart rate, chest pain type, exercise-induced angina, ST depression, and number of major vessels to predict whether a patient may have heart disease. These models should not replace a doctor, but they can help flag patients who may need further testing and help.

The models identified important patterns in features such as cholesterol, maximum heart rate, age, and ST depression. These features make sense medically because they are connected to cardiovascular health. Chest pain type and exercise-induced angina are also important because they relate to symptoms that may indicate heart problems.

Each model has a different use. Logistic Regression is helpful because it is simple and easy to interpret. KNN classifies patients by comparing them to similar patients in the dataset. Random Forest is useful because it can capture more complex patterns and show which features are most important.

However, there are limitations. The dataset is relatively small, so the results may not apply to all patient populations. The model could also be biased if certain groups are underrepresented in the data. False negatives are especially serious because the model could miss

a patient who actually has heart disease, while false positives could cause unnecessary stress or testing.

In the future, the model could be improved by using a larger and more diverse dataset. Adding features such as family history, smoking status, exercise habits, diabetes, and medication use could also improve accuracy.

Overall, these models can be useful as screening tools, but they should only support medical decisions and not replace professional diagnosis.

Conclusion

For this project, we used the UCI Heart Disease Dataset to predict whether a patient has heart disease based on medical and health-related features. We cleaned the data, removed missing values and irrelevant columns, split the dataset into training and testing sets, and compared three classification models: Logistic Regression, K-Nearest Neighbors, and Random Forest.

We evaluated the models using accuracy, precision, recall, F1-score, confusion matrices, and ROC-AUC. Since this is a healthcare-related problem, recall was especially important because it shows how well the model identifies patients who actually have heart disease. A model with low recall could miss patients who need medical attention, which makes false negatives a serious concern.

Overall, Random Forest appears to be the strongest model because it can capture more complex patterns in the data and provide feature importance, which helps explain which variables are most influential in predicting heart disease. Logistic Regression is still useful because it is simple, fast, and easy to interpret, while KNN is helpful because it classifies patients based on similarity to past cases.

In conclusion, machine learning can be a useful tool for supporting early heart disease screening. However, these models should not replace professional medical judgment. They should be used as decision-support tools to help identify patients who may need further testing or evaluation.